

Informe de refactorización — Fases 1, 2, 3 y 4

org.openmarkov.core y módulos dependientes

Manuel Arias Calleja

Marzo de 2026

Índice

1. Fase 1 — Seguridad, concurrencia y excepciones	2
1.1. Concurrencia en <code>PNESupport</code> : campo <code>listeners</code> (punto 1.1)	2
1.2. Concurrencia en <code>ProbNet</code> : campo <code>constantPotentials</code> (punto 1.2)	3
1.3. Rollback en <code>ProbNet.setNetworkType()</code> (punto 1.3)	3
1.4. Documentación del constructor de <code>ProbNet</code> (punto 1.4)	4
1.5. Concurrencia en <code>Node</code> : campo <code>potentials</code> (punto 1.5)	4
2. Fase 2 — Encapsulación	5
2.1. Campo <code>nodeType</code> de <code>Node</code> (punto 2.1)	5
2.2. Campos <code>states</code> y <code>variableType</code> de <code>Variable</code> (punto 2.1)	6
2.3. Campo <code>additionalProperties</code> en <code>ProbNet</code> , <code>Node</code> , <code>Variable</code> y <code>State</code> (punto 2.2)	6
2.4. Actualización de módulos externos (punto 2.3)	8
3. Fase 3 — Mejoras de API	9
3.1. Excepción semánticamente incorrecta en <code>AugmentedProbTable</code> (punto 3.1)	9
3.2. Validación de parámetros en <code>ProbNet.moveNode()</code> (punto 3.2)	10
3.3. Casts sin <code>instanceof</code> previo (punto 3.3)	10
3.4. Descomposición de <code>Variable.modifyState()</code> (punto 3.4)	11
4. Fase 4 — Eliminación de duplicación	12
4.1. Unificación de la lógica de copia en <code>ProbNetCopier</code> (punto 4.1)	12
4.2. Unificación de variantes <code>getPotentials</code> con <code>Predicate</code> (punto 4.2)	14
4.3. Creación de <code>UniformPotential</code> delegada en <code>PotentialOperations</code> (punto 4.3)	15
5. Tabla resumen de cambios	16
6. Conclusiones	16

Resumen ejecutivo

Este informe documenta las modificaciones realizadas en las fases 1 (*seguridad, concurrencia y excepciones*), 2 (*encapsulación*), 3 (*mejoras de API*) y 4 (*eliminación de duplicación*) de la campaña de refactorización sistemática de `org.openmarkov.core`. El trabajo afectó a las cuatro clases de dominio más fundamentales del módulo — `ProbNet`, `Node`, `Variable` y `State` — y propagó los cambios de API a ocho módulos dependientes.

Fase	Área	Ficheros modificados
1.1	Concurrencia: <code>PNESupport.listeners</code>	1
1.2	Concurrencia: <code>ProbNet.constantPotentials</code>	1
1.3	Rollback en <code>ProbNet.setNetworkType()</code>	1
1.4	Documentación del constructor de <code>ProbNet</code>	1
1.5	Concurrencia: <code>Node.potentials</code>	1
2.1	Encapsulación de campos en <code>Node</code> , <code>Variable</code> , <code>State</code>	3
2.2	Encapsulación y corrección de bugs en <code>ProbNet</code>	1
2.3	Actualización de módulos externos	≈30
3.1	Excepción incorrecta en <code>AugmentedProbTable</code>	1
3.2	Validación de parámetros en <code>ProbNet.moveNode()</code>	1
3.3	Casts sin <code>instanceof</code> en <code>ProbNet</code> y <code>Node</code>	2
3.4	Descomposición de <code>Variable.modifyState()</code>	1
4.1	Extracción de lógica de copia a <code>ProbNetCopier</code>	2
4.2	Unificación de variantes <code>getPotentials</code> con <code>Predicate</code>	2
4.3	Creación de <code>UniformPotential</code> delegada en <code>PotentialOperations</code>	1

Tabla 1: Resumen de actividad por subpunto.

1. Fase 1 — Seguridad, concurrencia y excepciones

1.1. Concurrencia en `PNESupport`: campo `listeners` (punto 1.1)

Estado anterior

```
private HashSet<PNEditListener> listeners = new HashSet<>();
```

El campo era un `HashSet` mutable no sincronizado que se exponía directamente a través del getter:

```
public HashSet<PNEditListener> getListeners() {  
    return listeners;  
}
```

Riesgos

- ConcurrentModificationException.** Si un hilo itera el conjunto en `notifyListeners()` mientras otro hilo invoca `addListener()` o `removeListener()`, el iterador de `HashSet` lanza `ConcurrentModificationException` inmediatamente porque detecta la modificación estructural del conjunto durante la iteración. Este tipo de fallo es no determinista y difícil de reproducir en pruebas.
- Exposición de la referencia interna.** Al devolver el propio `HashSet`, el código cliente podía añadir o eliminar listeners sin pasar por el contrato de `PNESupport`, eludiendo cualquier lógica que se añadiese en el futuro en los métodos `addListener` y `removeListener`.

- (c) **Reasignación directa del campo.** El método `setListeners()` reemplazaba la referencia del campo: si un hilo poseía una referencia al antiguo `HashSet`, seguiría usando el conjunto obsoleto; los nuevos eventos dejarían de llegarle.

Solución

```
private final Set<PNEditListener> listeners =
    ConcurrentHashMap.newKeySet();
```

- `ConcurrentHashMap.newKeySet()` produce un conjunto con la semántica de `Set` que soporta lecturas y escrituras concurrentes sin lanzar `ConcurrentModificationException` (los iteradores son *weakly consistent*).
- El campo se declara `final`: su identidad no cambia, solo su contenido.
- `getListeners()` devuelve `Collections.unmodifiableSet(listeners)`: vista de solo lectura que impide modificaciones externas sin pasar por los métodos de la clase.
- `setListeners()` realiza `clear()` + `addAll()` sobre el mismo conjunto en lugar de reasignar el campo.

1.2. Concurrencia en ProbNet: campo `constantPotentials` (punto 1.2)

Estado anterior

```
private HashSet<Potential> constantPotentials = new HashSet<>();
```

Riesgos

Idénticos al caso anterior: `HashSet` sin sincronización expuesto a través de un getter que devuelve la referencia directa. La diferencia es que `constantPotentials` se usa en operaciones de inferencia, que en versiones futuras del motor pueden ejecutarse en hilos de trabajo separados.

Solución

```
private final Set<Potential> constantPotentials =
    ConcurrentHashMap.newKeySet();
```

`getConstantPotentials()` pasa a devolver `Collections.unmodifiableSet(constantPotentials)`.

1.3. Rollback en `ProbNet.setNetworkType()` (punto 1.3)

Estado anterior

```
public void setNetworkType(NetworkType networkType) {
    NetworkType oldNetworkType = this.networkType;
    this.networkType = networkType;
    try {
        this.doEdit(new SetNetworkTypeEdit(this, networkType));
    } catch (Exception e) {
        throw new UnreachableException(e);
    }
}
```

La variable `oldNetworkType` se declaraba y asignaba pero nunca se usaba para ningún propósito. En caso de que `doEdit` lanzase una excepción (por ejemplo, porque la nueva restricción de tipo

de red viola alguna restricción de la estructura existente), el campo `this.networkType` ya había sido modificado antes de que se detectase el error.

Riesgo: objeto en estado inconsistente

Tras un fallo, `this.networkType` quedaría con el valor nuevo que no pudo validarse, mientras que el grafo contenido en `this` no habría cambiado para satisfacer las restricciones del nuevo tipo. Cualquier operación posterior que consultase el tipo de red obtendría un valor que no se corresponde con la estructura real del grafo.

Solución

```
public void setNetworkType(NetworkType networkType) {
    NetworkType oldNetworkType = this.networkType;
    this.networkType = networkType;
    try {
        this.doEdit(new SetNetworkTypeEdit(this, networkType));
    } catch (Exception e) {
        this.networkType = oldNetworkType;    // rollback
        throw new UnreachableException(e);
    }
}
```

Se añade la línea de rollback `this.networkType = oldNetworkType` en el bloque `catch`, de modo que si la validación falla el objeto queda exactamente como estaba antes de llamar al método.

1.4. Documentación del constructor de ProbNet (punto 1.4)

El constructor principal de `ProbNet` envuelve las posibles excepciones de `doEdit` en `UnreachableException`. Se añadió Javadoc explicando que se usa `ThrowableUtils.transferStackTrace` para preservar la cadena de causa y el *stack trace* original, evitando que la información de diagnóstico se pierda en el envoltorio.

1.5. Concurrencia en Node: campo potentials (punto 1.5)

Estado anterior

```
protected List<Potential> potentials;
// inicializado en el constructor como:
this.potentials = new ArrayList<>();
```

Los métodos `setPotential()` y `setPotentials()` realizaban operaciones compuestas sobre la lista:

```
public void setPotentials(List<Potential> potentials) {
    this.potentials.clear();
    this.potentials.addAll(potentials);
}
```

Riesgos

- (a) **ArrayList no es hilo-seguro.** Lecturas concurrentes desde código de inferencia y escrituras desde el hilo de edición pueden provocar `ArrayIndexOutOfBoundsException` o devolver una lista parcialmente actualizada.

- (b) **Operación compuesta no atómica.** Incluso usando `Collections.synchronizedList()`, la secuencia `clear() + addAll()` no sería atómica: entre ambas llamadas, otro hilo podría ver la lista vacía.

Solución

```
this.potentials = Collections.synchronizedList(new ArrayList<>());
```

Las operaciones simples sobre la lista (lectura individual, iteración en bloque propio del objeto) quedan protegidas. Para las operaciones compuestas se añade un bloque `synchronized` explícito:

```
public void setPotentials(List<Potential> potentials) {  
    synchronized (this.potentials) {  
        this.potentials.clear();  
        this.potentials.addAll(potentials);  
    }  
}
```

2. Fase 2 — Encapsulación

El objetivo de esta fase es eliminar los campos `public` y `protected` de las clases de dominio fundamentales, sustituyéndolos por campos `private` con métodos de acceso bien definidos. La motivación general es triple:

1. **Invariantes.** Los campos públicos pueden ser escritos desde cualquier punto del código sin ninguna validación, lo que hace imposible mantener invariantes de clase (por ejemplo, que el mapa de propiedades nunca sea `null`).
2. **Defensibilidad ante cambios.** Un campo público es una API pública; cambiar su tipo o semántica futura requiere modificar todos los accesos dispersos por el código base. Un método accessor puede evolucionar internamente sin alterar la firma.
3. **Copias defensivas.** Los arrays y colecciones expuestos directamente pueden ser modificados externamente, rompiendo el estado del objeto.

2.1. Campo `nodeType` de `Node` (punto 2.1)

Estado anterior

```
protected NodeType nodeType;
```

Problema

Al ser `protected`, las subclases y las clases del mismo paquete podían escribir directamente sobre `nodeType`, eludiendo cualquier control de acceso. Se encontraron accesos directos de escritura en `ProbNet` (líneas 907 y 1230) y en `DecisionTreeNode` (línea 174), todos ellos dentro del mismo proyecto.

Solución

```
private NodeType nodeType;
```

Todos los accesos directos se reemplazaron por llamadas a `getNodeType()`.

2.2. Campos `states` y `variableType` de `Variable` (punto 2.1)

Estado anterior

```
protected State[] states;  
protected VariableType variableType;
```

Problema con `states`

Un array Java es mutable por referencia: si `getStates()` devuelve `this.states`, el llamador puede escribir `states[i] = new State(...)` y modificar el estado interno del objeto sin que este sea consciente. El acceso `protected` además permitía a las subclases reasignar la referencia completa.

Solución

```
private State[] states;  
  
public State[] getStates() {  
    return states.clone();    // copia defensiva  
}  
  
public void setStates(State[] states) {  
    this.states = states.clone();    // copia defensiva  
}
```

Tanto el getter como el setter realizan una copia defensiva (`clone()`) para garantizar que el array interno no pueda ser modificado externamente.

`variableType` se hace `private` sin necesidad de copia defensiva porque `VariableType` es un `enum` (tipo por valor).

2.3. Campo `additionalProperties` en `ProbNet`, `Node`, `Variable` y `State` (punto 2.2)

Estado anterior

Las cuatro clases tenían un campo de propiedades arbitrarias con visibilidades inadecuadas:

Clase	Visibilidad anterior	Tipo anterior
<code>ProbNet</code>	<code>public</code>	<code>LinkedHashMap<String,String></code>
<code>Node</code>	<code>public</code>	<code>LinkedHashMap<String,String></code>
<code>Variable</code>	<code>protected</code>	<code>HashMap<String,String></code>
<code>State</code>	<code>public</code>	<code>LinkedHashMap<String,String></code>

Riesgos

- (a) **Mutación directa.** Cualquier código podía llamar a `x.additionalProperties.put(k, v)`, `x.additionalProperties.remove(k)` o incluso `x.additionalProperties = null`, rompiendo el invariante de que el mapa existe siempre.
- (b) **Semántica de copia rota.** Dado que el campo era público, el código de copia lo asignaba directamente:

```
newNode.additionalProperties = CloneUtils.safeClone(this.  
    additionalProperties);
```

Esto exige que quien hace la copia conozca el tipo concreto del campo (no la interfaz), e impide cambiar la implementación interna.

Bugs descubiertos durante la encapsulación

Al encapsular el campo se detectaron dos bugs en los métodos de copia de `ProbNet`:

Bug en `ProbNet.auxCopy()` (línea 550).

```
// Código buggy:
newNode.additionalProperties = additionalProperties;
//      ^-- campo de ProbNet, no del nodo que se copia
```

El código asignaba el mapa de propiedades de la *red* al nodo recién creado, en lugar de copiar las propiedades del *nodo* fuente. Esto hacía que todos los nodos de la copia compartiesen el mismo mapa de propiedades (el de la red), y que las propiedades específicas de cada nodo se perdiesen completamente. Además, al compartir la referencia, cualquier modificación a las propiedades de un nodo afectaría a todos los demás y a la propia red.

Bug en `ProbNet.addShiftedNode()` (línea 1348). Idéntico al anterior: el código copiaba el mapa de propiedades de la red en el nodo desplazado temporalmente, en lugar de copiar las propiedades del nodo original.

Corrección.

```
// Código corregido (ambos metodos):
newNode.setAdditionalProperties(node.getAdditionalProperties());
```

Solución general

Los cuatro campos se convierten a `private final Map<String,String>` inicializado en la declaración (o en el constructor cuando procede). Se añaden tres métodos en cada clase:

```
/** Vista no modificable del mapa de propiedades adicionales. */
public Map<String, String> getAdditionalProperties() {
    return Collections.unmodifiableMap(additionalProperties);
}

/** Reemplaza todo el contenido del mapa. */
public void setAdditionalProperties(Map<String, String> properties) {
    additionalProperties.clear();
    additionalProperties.putAll(properties);
}

/** Inserta o actualiza una propiedad individual. */
public void putAdditionalProperty(String key, String value) {
    additionalProperties.put(key, value);
}
```

Diseño del getter. Devolver `Collections.unmodifiableMap()` en lugar de una copia tiene coste cero y proporciona una vista en tiempo real del mapa que, sin embargo, no puede ser modificada por el llamador. Esto es preferible a una copia cuando el uso típico es solo de lectura (iterar propiedades al serializar o mostrar).

Diseño del setter. En lugar de reasignar la referencia del campo (imposible al ser `final`), se vacía el mapa existente y se vuelcan los nuevos pares. De este modo se preserva la invariante

de que el campo nunca es `null` y se mantiene la identidad del mapa (sin invalidar vistas no modificables existentes).

Estandarización del nombre. Los métodos anteriores se llamaban `getOtherProperties()` y `setOtherProperties()`, nombres inconsistentes con el nombre del campo `additionalProperties`. Se han renombrado a `getAdditionalProperties()` y `setAdditionalProperties()` respectivamente.

2.4. Actualización de módulos externos (punto 2.3)

La encapsulación de los campos `additionalProperties` en las clases de `core` rompió la compilación en ocho módulos dependientes. Se actualizaron todos los accesos directos al campo aplicando las siguientes transformaciones mecánicas:

Patrón antiguo	Patrón nuevo
<code>x.additionalProperties = m</code>	<code>x.setAdditionalProperties(m)</code>
<code>x.additionalProperties.put(k,v)</code>	<code>x.putAdditionalProperty(k,v)</code>
<code>x.additionalProperties.get(k)</code>	<code>x.getAdditionalProperties().get(k)</code>
<code>x.additionalProperties.keySet()</code>	<code>x.getAdditionalProperties().keySet()</code>
<code>x.additionalProperties.entrySet()</code>	<code>x.getAdditionalProperties().entrySet()</code>
<code>x.getOtherProperties()</code>	<code>x.getAdditionalProperties()</code>
<code>x.setOtherProperties(m)</code>	<code>x.setAdditionalProperties(m)</code>

Los módulos afectados fueron:

- `org.openmarkov.io`: `ElviraParser.java`, `ElviraWriter.java`, `ElviraUtil.java`, `PGMXReader_0_2.java`, `PGMXWriter_0_2.java` (con sus tests: `ElviraParserTest.java`, `ElviraParserSVTest.java`).
- `org.openmarkov.io.database.excel`: `OldExcelDataBaseIO.java`, `CSVDataBaseIO.java`, `ExcelDataBaseIO.java`.
- `org.openmarkov.io.database.weka`: `ArffParser.java`, `ArffDataBaseIO.java`.
- `org.openmarkov.io.database.elvira`: `ElviraDBParser.java`, `ElviraDataBaseIO.java`.
- `org.openmarkov.gui`: `PasteEdit.java`, `OtherPropertyEdit.java`, `PrefixedOtherPropertiesTablePan`.
- `org.openmarkov.learning.core`: `LearningManager.java` (tres líneas que leían el mapa mutable, añadían una clave y lo reasignaban se simplificaron a una única llamada `putAdditionalProperty("hasElviraProperties",zes)`).
- `org.openmarkov.integrationtests`: `Classificator.java` (accesos a `ProbNet.additionalProperties` y `State.additionalProperties`).

Caso especial: `ElviraWriter.generateElviraProperties()`. Este método obtenía una referencia directa al mapa de propiedades del nodo y lo mutaba:

```
// Código buggy:
Map<String, String> infoNode = node.additionalProperties;
// ... mutaciones sobre infoNode ...
```

Con el nuevo API, `getAdditionalProperties()` devuelve una vista no modificable, por lo que la mutación directa lanzaría `UnsupportedOperationException` en tiempo de ejecución. Se corrigió copiando primero el mapa, aplicando las mutaciones sobre la copia y finalmente persistiendo con `setAdditionalProperties()`:


```
Map<String, String> infoNode = new LinkedHashMap<>(node.
    getAdditionalProperties());
// ... mutaciones sobre infoNode ...
node.setAdditionalProperties(infoNode);
```

Caso especial: OtherPropertyEdit en el módulo GUI. Este editor de propiedades utilizaba `LinkedHashMap` como tipo concreto en su API interna (campos, parámetros y la llamada a `.clone()`). Se migró a la interfaz `Map<String,String>` y se sustituyeron las llamadas a `.clone()` por `new LinkedHashMap<>(...)`, que es la forma canónica de copiar un mapa:

```
// Antes:
this.newProperties = (LinkedHashMap<String,String>) this.oldProperties
    .clone();
// Despues:
this.newProperties = new LinkedHashMap<>(this.oldProperties);
```

3. Fase 3 — Mejoras de API

El objetivo de esta fase es corregir puntos de la API interna de `core` que presentan errores de contrato, ausencia de validación o complejidad excesiva, sin modificar las firmas públicas visibles desde los módulos dependientes.

3.1. Excepción semánticamente incorrecta en `AugmentedProbTable` (punto 3.1)

Estado anterior

```
} catch (NegativeArraySizeException e) {
    throw new OutOfMemoryError(e.getMessage());
}
```

Problemas

- Causa perdida.** `OutOfMemoryError` no expone un constructor (`Throwable cause`), por lo que la excepción original — incluyendo su *stack trace* y mensaje — desaparece por completo. Cualquier herramienta de diagnóstico o registro de errores verá solo el `OutOfMemoryError` con el mensaje copiado como cadena, sin poder navegar hasta el origen real del fallo.
- Tipo semánticamente incorrecto.** `OutOfMemoryError` indica que la JVM no puede asignar más memoria del *heap*. Un array de tamaño negativo es una violación de precondition (el argumento `tableSize` tiene un valor ilegal), no un agotamiento de recursos. Lanzar `OutOfMemoryError` induce a confusión: los sistemas de monitorización tratarán el evento como un fallo de memoria y no como un error de programación.

Solución

```
} catch (NegativeArraySizeException e) {
    throw new IllegalArgumentException(
        "Negative table size: " + tableSize, e);
}
```

`IllegalArgumentException` es el tipo estándar de Java para preconditiones violadas. El segundo argumento preserva la causa original en la cadena de excepciones.

3.2. Validación de parámetros en ProbNet.moveNode() (punto 3.2)

Estado anterior

```
public void moveNode(List<String> namesNode,
                    List<Point2D.Double> newPositions) {
    Node node;
    int i = 0;
    for (String name : namesNode) {
        node = getNode(name);
        node.setCoordinateX(newPositions.get(i).getX());
        node.setCoordinateY(newPositions.get(i).getY());
        i++;
    }
}
```

Problema

El método acepta dos listas que deben tener exactamente el mismo tamaño: la lista de nombres de nodo y la lista de posiciones nuevas. Si el llamador pasa listas de tamaños distintos, el bucle agota la lista más corta sin error (si `namesNode` es más corta) o lanza `IndexOutOfBoundsException` en `newPositions.get(i)` cuando `namesNode` es más larga. En ambos casos la red queda en un estado parcialmente actualizado sin ningún mensaje descriptivo que indique el origen del problema.

Solución

```
public void moveNode(List<String> namesNode,
                    List<Point2D.Double> newPositions) {
    if (namesNode.size() != newPositions.size()) {
        throw new IllegalArgumentException(
            "namesNode and newPositions must have the same size: "
            + namesNode.size() + " vs " + newPositions.size());
    }
    int i = 0;
    for (String name : namesNode) {
        Node node = getNode(name);
        node.setCoordinateX(newPositions.get(i).getX());
        node.setCoordinateY(newPositions.get(i).getY());
        i++;
    }
}
```

La validación al inicio del método es la forma canónica de expresar precondiciones en Java: falla rápido, con un mensaje que identifica el tamaño real de cada lista.

3.3. Casts sin instanceof previo (punto 3.3)

Se corrigieron dos *bare casts* en los que el compilador no puede garantizar el tipo en tiempo de compilación y un fallo producía `ClassCastException` sin contexto.

```
ProbNet.buildMarkovDecisionNetwork() — cast a TablePotential.
// Antes:
// TODO - Change constant potentials (Potential vs TablePotential)
this.constantPotentials.add((TablePotential) potential);

// Despues:
```

```

if (!(potential instanceof TablePotential tablePotential)) {
    throw new IllegalArgumentException(
        "Constant potential must be a TablePotential, got: "
        + potential.getClass().getName());
}
this.constantPotentials.add(tablePotential);

```

El TODO preexistente reconocía la deuda técnica de que `constantPotentials` almacena `TablePotential` aunque el campo que lo alimenta es de tipo `Potential`. Con el *pattern matching* se mantiene el comportamiento de fallo ante tipos inesperados pero con un mensaje que identifica la clase recibida.

`Node.absorbNodeConsistently()` — cast al resultado de `maximize()`.

```

// Antes:
TablePotential maximizedPotential =
    (TablePotential) DiscretePotentialOperations
        .maximize(utilityPotential, absorbedVariable)
        [0];

// Despues:
// maximize()[0] is always a TablePotential per its contract
Object[] maximizeResult = DiscretePotentialOperations
    .maximize(utilityPotential, absorbedVariable);
if (!(maximizeResult[0] instanceof TablePotential maximizedPotential))
{
    throw new IllegalStateException(
        "maximize() expected to return a TablePotential at index
        0, got: "
        + maximizeResult[0].getClass().getName());
}

```

`DiscretePotentialOperations.maximize(Potential, Variable)` devuelve `Object[]` (diseño heredado), donde el elemento 0 es siempre un `TablePotential` según el contrato del método. El *pattern matching* documenta este supuesto y lo convierte en un fallo explícito si alguna futura modificación de `maximize()` lo rompiese, en lugar de propagar un `ClassCastException` difícil de rastrear.

3.4. Descomposición de `Variable.modifyState()` (punto 3.4)

Estado anterior

`Variable.modifyState()` era un método de 134 líneas con un `switch` de cinco casos (ADD, REMOVE, DOWN, UP, RENAME), cada uno con su propia lógica de manipulación del array de estados y del `PartitionedInterval`. El único caller externo es `NodeStateEdit.executeEdit()`.

Problema

Un método de 134 líneas con cinco responsabilidades distintas viola el principio de responsabilidad única. El código comparte variables locales entre casos (`selectedStateIndex`, `oldStates`, `newState`) aunque cada caso usa un subconjunto diferente de ellas, lo que dificulta entender qué datos son relevantes para cada acción.

Solución

`modifyState()` se reduce a un despachador de 12 líneas que delega cada caso en un método privado:

```

public void modifyState(Node node, StateAction stateAction,
                        int stateIndex, String newName) {
    int selectedStateIndex = stateAction != StateAction.ADD
        ? getNumStates() - (stateIndex + 1) : 0;
    switch (stateAction) {
        case ADD      -> addStateAt(node, stateIndex, newName);
        case REMOVE   -> removeStateAt(node, selectedStateIndex);
        case DOWN     -> moveStateDown(node, selectedStateIndex);
        case UP       -> moveStateUp(node, selectedStateIndex);
        case RENAME   -> renameState(selectedStateIndex, newName);
    }
}

```

Los cinco métodos privados extraídos son:

- **addStateAt(Node, int, String)** — inserta el nuevo estado en la posición indicada, actualiza el potencial y, si la variable es **DISCRETIZED**, recalcula el **PartitionedInterval**.
- **removeStateAt(Node, int)** — elimina el estado en la posición indicada, actualiza el potencial y, si la variable es **NUMERIC** o **DISCRETIZED**, elimina el límite correspondiente del intervalo de partición.
- **moveStateDown(Node, int) / moveStateUp(Node, int)** — intercambian dos estados adyacentes y reordenan los potenciales del nodo y sus hijos.
- **renameState(int, String)** — renombra el objeto **State** en el índice indicado; como **getStates()** devuelve un clon del array pero los objetos **State** son referencias compartidas, la mutación actúa sobre el estado interno.

La firma pública de **modifyState()** no cambia; los callers existentes no se ven afectados.

4. Fase 4 — Eliminación de duplicación

El objetivo de esta fase es eliminar lógica repetida en las clases de dominio de **core**, extrayendo el código común a métodos o clases auxiliares bien nombrados. Las figuras 1 y 2 muestran únicamente los métodos afectados, antes y después de la refactorización.

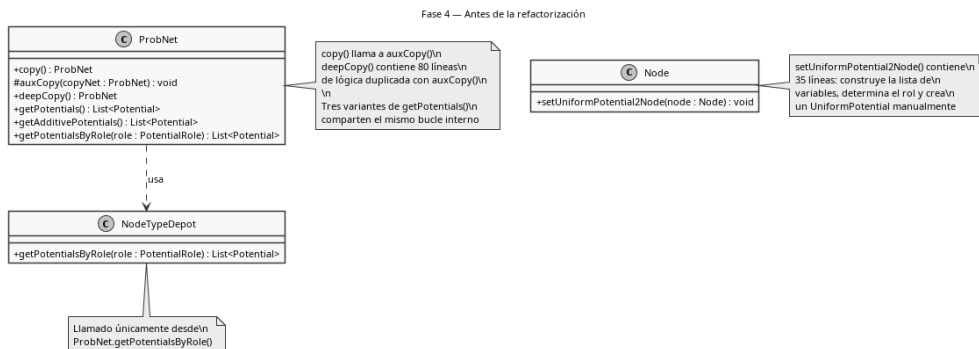


Figura 1: Diagrama de clases — estado antes de la Fase 4.

4.1. Unificación de la lógica de copia en ProbNetCopier (punto 4.1)

Estado anterior

ProbNet contenía dos métodos de copia con estructuras casi idénticas:

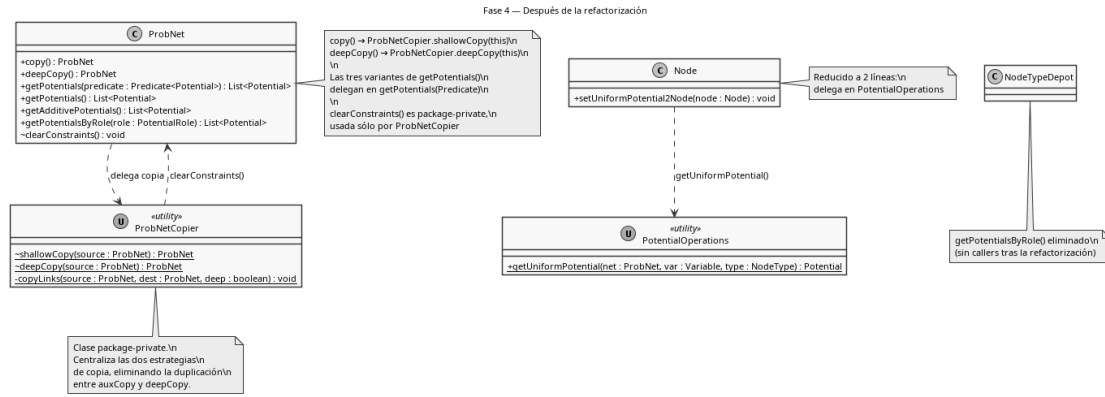


Figura 2: Diagrama de clases — estado después de la Fase 4.

```

// Copia superficial
protected ProbNet auxCopy(ProbNet copyNet) { ... } // ~75 líneas
public ProbNet copy() {
    return auxCopy(new ProbNet(this.networkType));
}

// Copia profunda
public ProbNet deepCopy() { ... } // ~80 líneas

```

Ambos métodos compartían el mismo esqueleto: copiar nombre, restricciones, nodos, enlaces, listeners y propiedades adicionales, diferenciándose solo en si los objetos (potenciales, nodos, intervalos) se compartían o se clonaban. El bloque de copia de enlaces en particular era casi idéntico línea a línea.

Riesgos

- Evolución divergente.** Cualquier corrección en la lógica de copia de enlaces (por ejemplo, añadir un nuevo campo a `Link`) requería modificar ambos métodos de forma independiente, con riesgo de olvidar uno de ellos.
- Visibilidad `protected` de `auxCopy`.** Al ser `protected`, el método formaba parte de la API de subclases aunque ninguna subclase de producción lo sobrescribía.

Solución

Se crea la clase de utilidad `ProbNetCopier` (misma package, acceso package-private) con dos métodos estáticos:

```

final class ProbNetCopier {
    static ProbNet shallowCopy(ProbNet source) { ... }
    static ProbNet deepCopy(ProbNet source) { ... }
    private static void copyLinks(ProbNet source, ProbNet dest,
                                  boolean deep) { ... }
}

```

El método privado `copyLinks` captura el único bucle que se repetía en ambas copias, parametrizando la profundidad mediante un booleano. `ProbNet` se reduce a delegaciones de una línea:

```

public ProbNet copy() {
    return ProbNetCopier.shallowCopy(this);
}

```

```

}

public ProbNet deepCopy() {
    return ProbNetCopier.deepCopy(this);
}

```

Se añade además el método package-private `ProbNet.clearConstraints()` para que `ProbNetCopier.deepCopy` pueda resetear el conjunto de restricciones sin acceder al campo privado directamente.

4.2. Unificación de variantes `getPotentials` con Predicate (punto 4.2)

Estado anterior

`ProbNet` y `NodeTypeDepot` contenían variantes del mismo bucle “iterar todos los nodos, filtrar sus potenciales y añadirlos a una lista”:

```

// Variante 1 - todos los potenciales no nulos
public List<Potential> getPotentials() {
    List<Potential> potentials = new ArrayList<>();
    for (Node node : getNodes()) {
        for (Potential p : node.getPotentials()) {
            if (p != null) potentials.add(p);
        }
    }
    for (Potential p : constantPotentials) {
        if (p != null) potentials.add(p);
    }
    return potentials;
}

// Variante 2 - solo los aditivos
public List<Potential> getAdditivePotentials() {
    List<Potential> potentials = new ArrayList<>();
    for (Node node : getNodes()) {
        for (Potential p : node.getPotentials()) {
            if (p != null && p.isAdditive()) potentials.add(p);
        }
    }
    // idem para constantPotentials...
}

// Variante 3 - por rol (en ProbNet, delegaba a NodeTypeDepot)
public List<Potential> getPotentialsByRole(PotentialRole role) {
    List<Potential> potentials = nodeDepot.getPotentialsByRole(role);
    for (Potential p : constantPotentials) {
        if (p.getPotentialRole() == role) potentials.add(p);
    }
    return potentials;
}

```

Solución

Se añade un método base parametrizado con `Predicate<Potential>`:

```

public List<Potential> getPotentials(Predicate<Potential> predicate) {
    List<Potential> potentials = new ArrayList<>();
    for (Node node : getNodes()) {

```

```

        for (Potential p : node.getPotentials()) {
            if (p != null && predicate.test(p)) potentials.add(p);
        }
    }
    for (Potential p : constantPotentials) {
        if (p != null && predicate.test(p)) potentials.add(p);
    }
    return potentials;
}

```

Las tres variantes se reducen a delegaciones de una línea:

```

public List<Potential> getPotentials() {
    return getPotentials(p -> true);
}

public List<Potential> getAdditivePotentials() {
    return getPotentials(Potential::isAdditive);
}

public List<Potential> getPotentialsByRole(PotentialRole role) {
    return getPotentials(p -> p.getPotentialRole() == role);
}

```

El método `getPotentialsByRole` de `NodeTypeDepot` queda sin callers y se elimina.

4.3. Creación de `UniformPotential` delegada en `PotentialOperations` (punto 4.3)

Estado anterior

`Node.setUniformPotential2Node()` construía manualmente la lista de variables (nodo + padres) y creaba un `UniformPotential`, lógica que `PotentialOperations.getUniformPotential()` ya encapsulaba:

```

public void setUniformPotential2Node(Node node) {
    List<Variable> variables = new ArrayList<>();
    PotentialRole role = node.getPotentials().getFirst().
        getPotentialRole();
    Variable thisVariable = node.getPotentials().getFirst().
        getVariable(0);
    variables.add(thisVariable);
    int numOfCellsInTable = thisVariable.getNumStates();
    double initialValue = Util.round(1.0 / numOfCellsInTable, "0.01");
    for (Node parent : node.getParents()) {
        variables.add(parent.getVariable());
        numOfCellsInTable *= parent.getVariable().getNumStates();
    }
    // TODO: This array is never used. Why is it created then?
    double[] table = new double[numOfCellsInTable];
    for (int i = 0; i < numOfCellsInTable; i++) table[i] =
        initialValue;

    UniformPotential uniformPotential = new UniformPotential(variables
        , role);
    node.setPotentials(new ArrayList<>(List.of(uniformPotential)));
}

```

El array `table` tiene incluso un `TODO` que reconoce que no se usa.

Solución

Se sustituye el cuerpo completo por una delegación de dos líneas:

```
public void setUniformPotential2Node(Node node) {
    Potential uniformPotential = PotentialOperations.
        getUniformPotential(
            node.getProbNet(), node.getVariable(), node.getNodeType())
        ;
    node.setPotentials(new ArrayList<>(List.of(uniformPotential)));
}
```

El array muerto desaparece junto con toda la construcción manual de la lista de variables.

5. Tabla resumen de cambios

6. Conclusiones

Las fases 1 y 2 de la refactorización eliminaron un conjunto de vulnerabilidades estructurales presentes en el corazón del modelo de dominio:

- **Dos bugs de copia** en `ProbNet` que hacían que los nodos copiados heredasen propiedades incorrectas (las de la red en lugar de las suyas propias), con posibilidad de interferencia entre nodos al compartir la misma referencia de mapa.
- **Un bug de estado inconsistente** en `setNetworkType()` que dejaba el objeto en un estado inválido tras un fallo de validación.
- **Tres puntos de carrera de datos** (*data race*) en colecciones no sincronizadas accesibles concurrentemente.
- **Múltiples violaciones de encapsulación** que permitían a código externo modificar el estado interno de las clases de dominio sin pasar por ninguna validación.

La fase 3 corrigió errores de contrato en la API interna: excepción semánticamente incorrecta, ausencia de validación de parámetros y casts sin comprobación de tipo previa.

La fase 4 ha reducido la duplicación estructural en `ProbNet` y `Node`:

- `ProbNetCopier` centraliza las dos estrategias de copia de red (superficial y profunda) que antes convivían como métodos paralelos en `ProbNet`, extrayendo el bucle de copia de enlaces como método privado compartido.
- `getPotentials(Predicate)` elimina tres variantes del mismo bucle de recolección de potenciales, permitiendo expresar cualquier filtro futuro en una sola línea.
- `Node.setUniformPotential2Node()` delega en `PotentialOperations.getUniformPotential()`, eliminando la duplicación con `Variable.setUniformPotential()` y borrando un array temporal que el propio código reconocía como innecesario.

El coste acumulado de las cuatro fases es la actualización de ≈ 50 puntos de uso en ocho módulos externos (fases 1–3) y la creación de un fichero nuevo (`ProbNetCopier.java`) junto con la eliminación de ≈ 160 líneas de código duplicado (fase 4).

Clase / fichero	Campo / método	Cambio
PNESupport	listeners	HashSet → ConcurrentHashMap.newKeySet(); final; getter retorna vista no modificable
ProbNet	constantPotentials	HashSet → ConcurrentHashMap.newKeySet(); final; getter retorna vista no modificable
ProbNet	setNetworkType()	Añadido rollback this.networkType = oldNetworkType en bloque catch
Node	potentials	ArrayList → Collections.synchronizedList; operaciones compuestas con synchronized
Node	nodeType	protected → private; accesos directos sustituidos por getNodeType()
Node	additionalProperties	public LinkedHashMap → private final Map; API: getAdditionalProperties(), setAdditionalProperties(), putAdditionalProperty()
Variable	states	protected State[] → private State[]; getter y setter con clone() defensivo
Variable	variableType	protected → private
Variable	additionalProperties	protected HashMap → private Map; misma API que Node
State	additionalProperties	public LinkedHashMap → private final Map; misma API que Node
ProbNet	additionalProperties	public LinkedHashMap → private final Map; misma API que Node
ProbNet.auxCopy()	línea 550	Bug: se copiaba el mapa de la red al nodo en lugar del mapa del nodo
ProbNet.addShiftedNode()	línea 1348	Bug: ídem al anterior
ProbNet	copy(), deepCopy()	Delegan en nueva clase ProbNetCopier; eliminado auxCopy()
ProbNetCopier (nueva)	shallowCopy(), deepCopy(), copyLinks()	Centraliza las dos estrategias de copia; copyLinks comparte el bucle de enlaces
ProbNet	getPotentials(Predictor)	Nuevo método base; getPotentials(), getAdditivePotentials() y getPotentialsByRole() delegan en él
NodeTypeDepot	getPotentialsByRole()	Eliminado (sin callers tras el punto 4.2)